

## Code-Layout, Readability and Reusability

|               |               |
|---------------|---------------|
| Date          | 01 July 2026  |
| Team ID       | XXXX          |
| Project Name  | Rising Waters |
| Maximum Marks | 5 Marks       |

### Code-Layout, Readability and Reusability:

This document evaluates the quality of the Rising Waters codebase in terms of its structure, readability, and reusability. The project is a Machine Learning-based flood prediction system built using Python, Flask, and classification algorithms (Decision Tree, Random Forest, KNN, and XGBoost). Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

### Code Layout Checklist:

| S.No | Code Quality Parameter    | Description                                   | Followed (Yes/No/Partial) | Remarks                                                                                             |
|------|---------------------------|-----------------------------------------------|---------------------------|-----------------------------------------------------------------------------------------------------|
| 1    | Consistent Indentation    | Uniform spacing/tabs used throughout the code | Yes                       | PEP8-style 4-space indentation followed across Jupyter notebook and Flask app.py.                   |
| 2    | Proper File Structure     | Files and folders are logically organized     | Yes                       | Separate folders for dataset, model (floods.save), templates, static files, and app.py.             |
| 3    | Meaningful Variable Names | Variables reflect their purpose clearly       | Yes                       | Variables like rainfall, cloud_visibility, seasonal_rainfall and y_pred used consistently.          |
| 4    | Function / Method Names   | Functions are descriptively named             | Yes                       | Functions such as preprocess_data(), train_model(), predict_flood() clearly indicate purpose.       |
| 5    | Code Comments             | Inline and block comments explain logic       | Partial                   | Key preprocessing and model training steps are commented; some Flask routes need more comments.     |
| 6    | Modular Design            | Code is split into reusable functions/modules | Yes                       | Data preprocessing, model training, and Flask routing are separated into distinct modules.          |
| 7    | No Redundant Code         | Duplicate or unused code is removed           | Yes                       | Notebook cleaned up after model comparison; only the final XGBoost pipeline retained in app.py.     |
| 8    | Error Handling            | Exceptions and errors are handled gracefully  | Partial                   | Try/except used for form input validation; additional handling recommended for model load failures. |

### Reusable Components / Modules:

| S.No | Component / Module Name              | Language / Technology                | Where Reused                                                                                        | Reusability Level (High/Medium/Low) |
|------|--------------------------------------|--------------------------------------|-----------------------------------------------------------------------------------------------------|-------------------------------------|
| 1    | Data Preprocessing Module            | Python (Pandas, NumPy, Scikit-learn) | Used for cleaning, encoding, and scaling rainfall/weather data before training all four models.     | High                                |
| 2    | Model Training & Evaluation Function | Python (Scikit-learn, XGBoost)       | Reused to train and evaluate Decision Tree, Random Forest, KNN, and XGBoost with the same pipeline. | High                                |
| 3    | Trained Model File (floods.save)     | Pickle / Joblib                      | Loaded in the Flask app to generate flood predictions from user input.                              | High                                |
| 4    | Flask Routing & Form Handling        | Flask, HTML, CSS, JavaScript         | Reused across Home Page, Prediction Input Page, and Result Pages.                                   | Medium                              |
| 5    | Visualization Utility Functions      | Matplotlib, Seaborn                  | Reused for univariate/multivariate analysis, distribution plots, box plots, and heat maps.          | Medium                              |

### Overall Code Quality Assessment

| Aspect                   | Rating (1-5) | Comments                                                                        |
|--------------------------|--------------|---------------------------------------------------------------------------------|
| Code Layout & Structure  | 4            | Clean, logically organized notebook and Flask project structure.                |
| Readability              | 4            | Descriptive variable and function names make the flow easy to follow.           |
| Reusability              | 5            | Preprocessing and model training pipeline is reused across all four algorithms. |
| Documentation / Comments | 3            | Core logic is commented; route handlers could use more detail.                  |
| <b>Overall Score</b>     | 4            | Well-structured, maintainable codebase suitable for future extension.           |